

1. **Why version control:** What version control is, and why · Git vs GitHub · Repository
2. **How Git thinks:** The three areas (and the remote) · The staging area (index) · Commits: snapshot, hash, message
3. **Setup:** Install and configure Git · Signing in to GitHub from Git · git init vs git clone
4. **Everyday commands:** The daily loop: status, add, commit · Inspecting: git log and git diff · .gitignore: what never goes into Git
5. **Remotes:** Remotes, origin and git push · git fetch vs git pull
6. **Branches and merging:** Branches and HEAD · Merging: fast-forward vs merge commit · Merge conflicts
7. **Working in a team:** GitHub flow and pull requests
8. **Undoing safely:** Undo before committing: git restore · Undo after committing: revert, amend, reset

Why version control

What version control is, and why

Version control is a system that records chosen, labelled states of a whole project over time, so you can see what changed, when and by whom, return to any earlier state, and work in parallel with other people.

Example. On Monday you save a checkpoint called "Add contact form". On Wednesday the layout is broken. The history shows three checkpoints since Monday, and comparing Monday with today shows the single line in `styles.css` that changed `display: grid` to `display: flex`. You fix that line instead of guessing.

Common mistake. Treating version control as cloud sync (OneDrive, Dropbox, Google Drive). Sync silently keeps the **latest** version of each file; version control keeps **chosen, described, whole-project** checkpoints, and records nothing until you ask it to. Do not keep a Git project inside a synced folder either: sync and Git writing the same hidden files at the same time causes lock errors and damaged repositories.

Git vs GitHub

Git is the version-control program that runs on your computer; GitHub is a website that stores copies of Git repositories online and adds team features such as pull requests, reviews and issues.

Who does what

	Git	GitHub
What it is	A command-line program you install	A website (a hosting service) run by a company
Where it runs	Your laptop, offline	GitHub servers, through the browser or the network
Main jobs	Stage, commit, branch, merge, read history	Store a shared copy, pull requests, code review, access control
Account needed?	No (only a name and email in its config)	Yes, a free github.com account

Example. On a train without Wi-Fi you create a repository, make 12 commits and browse the history: that is all Git. At home, `git push` uploads those 12 commits to GitHub, where your teammate sees them and can review them.

Common mistake. "I committed, so my instructor can see it." A commit exists only in your **local** repository until you run `git push`. The mirror mistake: editing a file on github.com and expecting your laptop to know about it; your local copy changes only after `git pull`.

Repository

A repository (repo) is a project folder whose complete history Git tracks; the history and Git's settings live in a hidden `.git` folder at the root of that folder.

shell

```
$ cd my-first-repo
$ ls -a
.  ..  .git  README.md  index.html
$ git status
On branch main
Your branch is up to date with 'origin/main'.
```

nothing to commit, working tree clean

Example. A two-person project lives in one repository on GitHub. Each partner has a local repository created by cloning it. Both local repositories hold the full history, so if one laptop dies, nothing committed and pushed is lost.

Common mistake. Running `git init` in your home folder or on the Desktop, which turns every file below it into one giant repository, or running it inside a folder that is already a repository (a repository nested in another). Check where you are with `pwd` and `git status` before `git init`: if `git status` already answers "On branch ...", you are inside a repository.

How Git thinks

The three areas (and the remote)

Git keeps your work in three local areas: the **working directory** (the files you edit), the **staging area** or **index** (the next commit being prepared) and the **repository** (the commits already saved); a remote such as GitHub is a fourth, separate copy.

The four places your work can be

Area	What it holds	Where it lives	In / out
Working directory	The files as you see them in the editor, including unsaved-to-Git experiments	The project folder	You edit; <code>git restore</code> discards edits
Staging area (index)	The exact content of the next commit	A file inside <code>.git</code>	<code>git add</code> in; <code>git restore --staged</code> out
Local repository	Every commit, every branch	The <code>.git</code> folder	<code>git commit</code> in
Remote (GitHub)	The shared copy of the commits	A server	<code>git push</code> up; <code>git fetch</code> / <code>git pull</code> down

diagram

```

working directory  --git add-->  staging area  --git commit-->  local repo  --git push-->  remote
  (your desk)                (the box)                (the shelf)                (GitHub)
      <--git restore--                <--git restore --staged                <--git pull--

```

Example. You create `about.html` (untracked) and edit `index.html` (modified). `git add about.html` puts only the new page in the box. `git commit -m "Add about page"` seals it: the about page is now in the repository, while `index.html` is still modified on the desk and in no commit. GitHub has none of this until `git push`.

Common mistake. Believing that pressing Ctrl+S in VS Code "saves into Git". Saving writes only to the **working directory**. Until you `git add` and `git commit`, Git has no copy of that change, and until you `git push`, nobody else has one either.

The staging area (index)

The **staging area**, also called the **index**, is a draft of your next commit: it holds the exact version of each file that `git commit` will record.

shell

```

$ git add styles.css      # photo of styles.css goes into the draft
$ # ...you edit styles.css again and save...
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   styles.css

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)

```

(use "git restore <file>..." to discard changes in working directory)
modified: styles.css

Example. You fix a typo in the heading of index.html and, in the same session, start an experimental animation in styles.css. git add index.html then git commit -m "Fix typo in main heading" records only the fix. The animation stays modified in the working directory, out of history, until it works.

Common mistake. Thinking that once a file has been added it is "in Git" with whatever content it has later. git add stages the content **at that moment**; edits made afterwards need another git add, or the commit silently contains the older version. Also, git add does not move the file anywhere: it copies its content, and the file stays on your disk.

Commits: snapshot, hash, message

A commit is a saved snapshot of the whole project at one moment, identified by a unique **hash** and stored with an author, a date, a message and a link to its parent commit(s).

Commit messages

Weak	Better	Why
update	Add footer with contact links	Says what the commit does
fix	Fix 400 response when title is empty	Says which bug, so it can be found later
final version!!	Validate year as an integer in PATCH	Nothing is ever final; describe the change
stuff from today	Two commits: Add /stats route, Style the book list	One logical change per commit

shell

```
$ git log
commit 3f9c2a1e8b7d4c6a5f0e1d2c3b4a59687f1e2d3c (HEAD -> main, origin/main)
Author: Ana Ruiz <ana.ruiz@example.com>
Date: Mon Oct 5 10:12:44 2026 +0200
```

Add contact form to index.html

Example. A reviewer, a teammate or you three months later asks: "which commit fixed the bug where PATCH wiped other fields?" A history like a1b2c3d Add Book store module, e4f5a6b Validate title and year, c7d8e9f Fix PATCH overwriting untouched fields lets you find and link that fix in seconds; a history of 40 commits called "changes" does not.

Common mistake. Saving all work in one huge commit at the end ("final"). You lose the ability to undo one change without the others, your partner cannot review it, and the history no longer shows who did what or why. Commit small and often: each time one thing works.

Setup

Install and configure Git

Install Git once per computer, then tell it who you are (the name and email stamped on every commit), to call the first branch main, and which editor to open when it needs a message.

shell

```
# identity: use the same email as your GitHub account
git config --global user.name "Ana Ruiz"
git config --global user.email "ana.ruiz@example.com"

# name the first branch main, like GitHub
git config --global init.defaultBranch main

# open VS Code (not Vim) when Git needs a message, e.g. for a merge commit
git config --global core.editor "code --wait"

# check everything
git config --list
```

Example. Ana sets her name and the email of her GitHub account on her laptop. Every commit she makes in the team project shows "Ana Ruiz" with her avatar on GitHub, so anyone reading the history can see the work of both partners. Her partner forgot the email step on a shared computer: his commits appear on GitHub as an unlinked author, with no profile picture.

Common mistake. Skipping the configuration. Git then refuses to commit with "Please tell me who you are", or (on some systems) guesses an identity like `ana@LAPTOP-7Q2.local` that GitHub cannot link to your account. A second classic: without `core.editor`, a merge opens **Vim** and you cannot get out. In Vim, type `:wq` and press Enter to save and quit.

Signing in to GitHub from Git

GitHub does not accept your account password for Git commands; you prove who you are with a browser sign-in through a credential helper, a personal access token, the GitHub CLI or an SSH key.

Ways to authenticate

Option	How	Notes
Git Credential Manager (HTTPS)	Included with Git for Windows. On the first push a browser window opens; sign in to GitHub and approve	Nothing to configure on Windows
GitHub CLI (HTTPS)	Install <code>gh</code> , run <code>gh auth login</code> , choose HTTPS and "Login with a web browser"	Works on Windows, macOS and Linux; also configures Git
Personal access token (HTTPS)	<code>github.com</code> → Settings → Developer settings → Personal access tokens → generate one; paste it when Git asks for a password	Treat it like a password; it expires; never commit it
SSH key	<code>ssh-keygen -t ed25519 -C "you@example.com"</code> , add the public key (<code>~/.ssh/id_ed25519.pub</code>) in Settings → SSH and GPG keys; use the SSH URL <code>git@github.com:user/repo.git</code>	No prompts afterwards; the private key never leaves your computer

shell

```
$ git push
Username for 'https://github.com': ana-ruiz
Password for 'https://ana-ruiz@github.com':      # account password typed here
remote: Support for password authentication was removed on August 13, 2021.
fatal: Authentication failed for 'https://github.com/ana-ruiz/portfolio.git/'

$ gh auth login      # fix: sign in once through the browser, then push again
$ git push
```

Example. On a Windows laptop with Git for Windows, Ana's first `git push` opens the browser, she clicks "Authorize", and the push completes. On a shared Linux machine the same push asks for a username and password, so she runs `gh auth login` once instead.

Common mistake. Pasting a token into a script, a README or a `.env` that gets committed. A token in a public repository is a leaked key: anyone can push to your repositories with it. If it happens, delete the token on GitHub immediately and generate a new one.

git init vs git clone

`git init` turns a local folder into a new, empty repository with no history and no remote; `git clone <url>` downloads an existing repository, with all its files and history, and links it to its source under the name `origin`.

Which one?

	<code>git init</code>	<code>git clone <url></code>
Starts from	A local folder	A repository that already exists (usually on GitHub)
History	Empty	Complete copy
Remote	None: add with <code>git remote add origin <url></code>	<code>origin</code> set up automatically
First push	<code>git push -u origin main</code>	<code>git push</code>
Run it	Inside the project folder	In the parent folder: it creates the project folder

shell

```
# Path A (recommended): repository created on GitHub first
git clone https://github.com/ana-ruiz/books-api.git
cd books-api

# Path B: existing local folder, EMPTY repository created on GitHub
cd my-project
git init
git add .
git commit -m "Initial commit"
git remote add origin https://github.com/ana-ruiz/my-project.git
git push -u origin main
```

Example. For a team project, one partner creates `books-api` on GitHub with a README and adds the other as a collaborator; then **both** run `git clone` on their own laptops. Both local repositories share the same first commit, so their work can later be pushed, pulled and merged.

Common mistake. Doing both: creating the GitHub repository **with** a README and also running `git init` and committing locally. The two repositories now have unrelated first commits, `git push` is rejected and `git pull` stops with "refusing to merge unrelated histories". Start again by cloning, and copy your files into the clone.

Everyday commands

The daily loop: status, add, commit

The everyday loop is: edit files, `git status`, `git add` the changes that belong together, `git commit -m "message"`, and repeat; `git status` tells you at every step which area each file is in.

shell

```
$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   index.html

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        about.html

no changes added to commit (use "git add" and/or "git commit -a")

$ git add index.html about.html
$ git commit -m "Add about page and link it from the home page"
[main 4b1e9d2] Add about page and link it from the home page
 2 files changed, 31 insertions(+), 1 deletion(-)
 create mode 100644 about.html

$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
```

Example. Read the last `git status` above: "ahead of 'origin/main' by 1 commit" means the commit exists only on your laptop. The loop is finished for GitHub only after `git push`.

Common mistake. Running `git add .` without looking. It happily stages `node_modules/` (thousands of files) and `.env` (your secrets) if no `.gitignore` excludes them. Always run `git status` after `git add .` and before `git commit`, and read the list.

Inspecting: git log and git diff

`git log` **lists the commits of the current branch, newest first**; `git diff` **shows line by line what you changed but have not staged**, and `git diff --staged` **shows what the next commit will contain**.

shell

```
$ git diff
diff --git a/index.html b/index.html
index 8a1f2c3..5d7e9b0 100644
--- a/index.html
+++ b/index.html
@@ -6,7 +6,7 @@
<body>
  <header>
    <a href="index.html">Home</a>
-   <h1>My Portfolio</h1>
+   <h1>Ana Ruiz - Portfolio</h1>
    <p>Web developer</p>
  </header>
  <main>

$ git log --oneline
4b1e9d2 (HEAD -> main) Add about page and link it from the home page
3f9c2a1 (origin/main) Add contact form to index.html
a07e5c8 Initial commit
```

Example. Before committing, run `git diff --staged` and read it as your partner would: if you see a `console.log("here")` you forgot to remove, take it out now. The `--oneline` log above also shows, through the labels, that `main` is one commit ahead of `origin/main`.

Common mistake. Running `git diff` right after `git add` and seeing nothing, then concluding the changes are lost. They are safe in the staging area; plain `git diff` only shows **unstaged** edits. Use `git diff --staged` to see them.

.gitignore: what never goes into Git

A **.gitignore** file in the repository root lists file patterns Git must not track, such as `node_modules/` (re-created by `npm install`) and `.env` (secrets).

gitignore

```
# dependencies: re-created by npm install
node_modules/

# secrets: never commit; ship .env.example with fake values instead
.env
.env.*
!.env.example

# build output and test reports
dist/
coverage/

# operating-system clutter
.DS_Store
Thumbs.db
```

Example. An API reads `DATABASE_URL` and `JWT_SECRET` from `.env`, ships a `.env.example`, and must run "from a clean clone": a reviewer or a new teammate clones it, runs `npm install` (which rebuilds `node_modules/`) and copies `.env.example` to `.env`. A committed `.env` with a real secret is a leak: anyone who can read the repository (or its history) has the key. The `.gitignore` above covers all of that.

Common mistake. Adding `.env` to `.gitignore` **after** it was committed and pushed. Git keeps tracking it, and the secret stays readable in the history on GitHub. Fix: `git rm --cached .env`, commit, push, and **change the secret** (new password or key), because the old value is public for ever in the history.

Remotes

Remotes, origin and git push

A **remote** is a named link to another copy of the repository, usually on GitHub (origin is the name `git clone` gives it); `git push` uploads the commits of your current branch that the remote does not have yet.

shell

```
$ git remote -v
origin https://github.com/ana-ruiz/books-api.git (fetch)
origin https://github.com/ana-ruiz/books-api.git (push)

$ git push -u origin main      # first push of main
$ git push                    # later pushes

# when your partner pushed first:
$ git push
To https://github.com/ana-ruiz/books-api.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://github.com/ana-ruiz/books-api.git'
$ git pull                    # integrate their commits, then
$ git push
```

Example. Reviewers, teammates and deployment tools read the default branch **on GitHub**, not your laptop. After the final `git push`, open the repository page on github.com and check that the latest commit message and time are yours.

Common mistake. Trusting "Your branch is up to date with 'origin/main'" as live news. It compares with origin/main **as of your last fetch or pull**; your partner may have pushed five minutes ago. Run `git fetch` (or `git pull`) first if you need the current truth.

git fetch vs git pull

`git fetch` **downloads new commits from the remote into** `origin/<branch>` **without touching your files or your branch**; `git pull` **is** `git fetch` **followed by integrating those commits into your current branch (by default with a merge).**

fetch vs pull

	git fetch	git pull
Downloads new commits	Yes	Yes
Updates <code>origin/main</code>	Yes	Yes
Changes your branch and files	No	Yes (merge, or fast-forward)
Can cause a conflict	Never	Yes, if both sides changed the same lines

shell

```
# once per computer: make pull merge when histories diverge
git config --global pull.rebase false

git fetch                # see what arrived without changing anything
git log --oneline main..origin/main # commits on GitHub you do not have yet
git pull                 # bring them into your branch
```

Example. Monday morning: your partner pushed the validation module during the weekend. `git pull` brings their three commits into your `main` (a fast-forward, because you had not committed anything since). Only then do you start your own work.

Common mistake. Pulling with uncommitted edits to the same files the incoming commits change. Git refuses with "Your local changes to the following files would be overwritten by merge". Commit your work (or discard it with `git restore`) and pull again.

Branches and merging

Branches and HEAD

A branch is a **movable label that points to one commit**; when you commit on a branch, its label moves forward to the new commit. HEAD marks the branch you are currently on.

Branch commands

Command	What it does
<code>git branch</code>	Lists local branches; * marks the current one
<code>git branch add-footer</code>	Creates the label at the current commit; you stay on your branch
<code>git switch add-footer</code>	Moves HEAD to that branch and updates your files (older: <code>git checkout add-footer</code>)
<code>git switch -c add-footer</code>	Creates and switches in one step (older: <code>git checkout -b add-footer</code>)
<code>git branch -d add-footer</code>	Deletes a branch that is already merged (the commits remain)

shell

```
$ git switch -c add-footer
Switched to a new branch 'add-footer'
$ git branch
* add-footer
  main
$ # ...edit index.html and save...
$ git commit -am "Add footer"      # -a stages already-tracked modified files
$ git log --oneline --graph --all
* 8e4d7b2 (HEAD -> add-footer) Add footer
* 3f9c2a1 (origin/main, main) Add contact form to index.html
```

Example. Before the commit above, `main` and `add-footer` both pointed at `3f9c2a1`. After it, only `add-footer` moved to `8e4d7b2`; `main` still points at `3f9c2a1`, so switching back to `main` shows the page without the footer.

Common mistake. Creating a branch with `git branch add-footer` and then committing without switching: the commits land on `main`, because HEAD never moved. Check the * in `git branch` (or the first line of `git status`, "On branch ...") before you commit. Also: commit before switching; uncommitted edits travel with you to the other branch, or Git refuses to switch if they would be overwritten.

Merging: fast-forward vs merge commit

`git merge <branch>` **brings the commits of <branch> into the branch you are on**; if your branch has not moved since the other one started, Git just slides your label forward (**fast-forward**), otherwise it creates a **merge commit with two parents**.

shell

```
$ git switch main
$ git merge add-footer      # main had not moved: fast-forward
Updating 3f9c2a1..8e4d7b2
Fast-forward
 index.html | 6 +++++
 1 file changed, 6 insertions(+)

$ git merge add-search      # main had moved too: merge commit
Merge made by the 'ort' strategy.
 app.js | 14 ++++++++
 1 file changed, 14 insertions(+)

$ git log --oneline --graph
* d41c9a7 (HEAD -> main) Merge branch 'add-search'
|\
| * 5b2e1f0 (add-search) Add search box
* | 8e4d7b2 (add-footer) Add footer
|/
* 3f9c2a1 Add contact form to index.html
```

Example. Ana branches `add-search` from `3f9c2a1`. Meanwhile her footer branch is fast-forwarded into `main`, so `main` moves to `8e4d7b2`. When she merges `add-search`, both sides have new commits, so Git creates the merge commit `d41c9a7`, whose parents are `8e4d7b2` and `5b2e1f0`.

Common mistake. Merging in the wrong direction: running `git merge main` while on `add-footer` when you meant to bring the footer into `main`. That updates the feature branch instead. Read "On branch ..." in `git status` first: you must be **on the receiving branch**.

Merge conflicts

A merge conflict happens when both sides changed the **same lines** of a file (or one side deleted a file the other edited); Git stops, writes both versions into the file between markers, and waits for you to choose the final content and commit.

shell

```
$ git merge add-title
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.

# index.html now contains:
<<<<<< HEAD
    <h1>Ana Ruiz - Portfolio</h1>
=====
    <h1>Ana Ruiz - Web Developer</h1>
>>>>>> add-title

# edit it to the final line, e.g. <h1>Ana Ruiz - Web Developer Portfolio</h1>
$ git add index.html
$ git commit                # accept the pre-filled "Merge branch 'add-title'"
```

Example. In a team API project both partners edit the endpoint table in `README.md`. Luis pushes first; Ana's `git pull` stops with a conflict in `README.md`. She keeps both new rows, deletes the markers, runs `git add README.md` and `git commit`, then `git push`. Total: two minutes.

Common mistake. Committing with the markers still in the file. The page then shows `<<<<<< HEAD`, or JavaScript fails with a syntax error. Before `git add`, search the file for `<<<<<<`. The other panic reaction, deleting the folder and cloning again, throws away your un-pushed commits.

Working in a team

GitHub flow and pull requests

GitHub flow is a team workflow: create a branch for each change, push it, open a **pull request (PR)** on GitHub, get it reviewed, merge it into `main`, and everyone pulls the updated `main`.

1. `git switch main` and `git pull` (start from the latest `main`).
2. `git switch -c validate-form` (one branch per change, named after it).
3. Edit, `git add`, `git commit`: small commits.
4. `git push -u origin validate-form`.
5. On GitHub: **Compare & pull request**, describe what and why, request your partner as reviewer.
6. The reviewer reads the diff, comments or approves; you push fixes to the same branch and the PR updates itself.
7. **Merge pull request** on GitHub, then locally `git switch main`, `git pull`, `git branch -d validate-form`.

shell

```
git switch main
git pull
git switch -c validate-form
# ...edit, git add, git commit (several small commits)...
git push -u origin validate-form
# on github.com: Compare & pull request -> request review -> Merge pull request
```

```
git switch main
git pull
git branch -d validate-form
```

Example. A project reaches a first finished version and you want to start the next phase while keeping that version reachable. The simplest way is a branch label on its last commit: `git branch v1-final` then `git push origin v1-final`, and keep working on `main` through pull requests. The README then says: "The first version is on the `v1-final` branch".

Common mistake. Long-lived branches: two weeks of work on one branch without pulling `main` produces one enormous PR with dozens of conflicts. Keep each PR to one change and merge it within a day or two. And never keep committing on a branch whose PR was already merged; start a new branch from the updated `main`.

Undoing safely

Undo before committing: `git restore`

`git restore --staged <file>` **takes a file out of the staging area but keeps your edits**; `git restore <file>` **throws away uncommitted edits in the working directory, returning the file to its staged (or last committed) version.**

Pick the right undo

Situation	Command	Safe?
Staged a file by mistake (e.g. <code>.env</code>)	<code>git restore --staged .env</code>	Yes: the file and its content stay
Want to throw away an unstaged experiment	<code>git restore styles.css</code>	No: the edits are lost for good
Staged and want to discard	<code>git restore --staged styles.css</code> then <code>git restore styles.css</code>	No: the edits are lost
Deleted a tracked file by accident	<code>git restore index.html</code>	Yes: it comes back from Git

shell

```
$ git add .
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   .env
    modified:   index.html

$ git restore --staged .env      # .env back to untracked; the file is untouched
$ git commit -m "Show book count in header"

$ git restore styles.css        # DISCARD the unstaged edits to styles.css
```

Example. You ran `git add .` and `git status` shows `new file: .env`. Before committing, `git restore --staged .env` takes it out; then add `.env` to `.gitignore` so it never happens again. Nothing has left your laptop, so nothing leaked.

Common mistake. Typing `git restore styles.css` when you meant `git restore --staged styles.css`. The first one erases a morning of uncommitted work with no way back. When unsure, commit first (on a branch if it is experimental): committed work can always be recovered.

Undo after committing: `revert`, `amend`, `reset`

To undo a commit that is already pushed, use `git revert <hash>`, which adds a new commit that cancels it; `git commit --amend` and `git reset` rewrite history, so use them only on commits that exist only on your laptop.

Undoing commits (intro level)

Command	What it does	Rewrites history?	OK after push?
<code>git revert <hash></code>	New commit with the opposite changes	No	Yes
<code>git commit --amend</code>	Replaces the last commit (fix message, add a forgotten file)	Yes	No
<code>git reset --soft HEAD~1</code>	Removes the last commit; its changes stay staged	Yes	No

Command	What it does	Rewrites history?	OK after push?
<code>git reset --hard <hash></code>	Moves the branch back and discards all uncommitted work	Yes	No; avoid at this level

shell

```
# a pushed commit broke the page: cancel it with a new commit
$ git log --oneline
8e4d7b2 (HEAD -> main, origin/main) Remove contact form
3f9c2a1 Add contact form to index.html
$ git revert --no-edit 8e4d7b2
[main c71a0f4] Revert "Remove contact form"
 1 file changed, 12 insertions(+)
$ git push

# forgot a file in the last commit, NOT pushed yet:
$ git add about.html
$ git commit --amend --no-edit
```

Example. Luis pushed a commit that deleted the contact form. Ana, who already pulled it, sees `git revert` arrive as an ordinary new commit "Revert ..." on her next pull. Had Luis used `git reset` and `git push --force`, Ana's next push would resurrect the bad commit or fail, and work could be lost.

Common mistake. Amending or resetting a commit that was already pushed, then "fixing" the rejected push with `git push --force`. That replaces the shared history and can delete your partner's commits. On a shared branch, use `git revert`.